

DOCUMENTATION TECHNIQUE

DCLIC - Application de Prise de Notes (Flutter & SQLite)

🔗 Conformité à l'énoncé (Semaine 6)

Écran de connexion obligatoire (email + mdp).

Message d'erreur clair et précis en cas d'échec.

Écran principal intuitif avec liste des notes.

Gestion complète : ajout, édition, et suppression.

Données persistées localement et de manière transparente grâce à SQLite.

1. Fonctionnalités & Ergonomie

1.1 Interface de connexion scrollable

- **Saisie sécurisée** : Champs dédiés pour l'adresse email et le mot de passe (masqué).
- **Protection anti-overflow** : L'ensemble du formulaire est enveloppé dans un `SingleChildScrollView`. Lorsque le clavier virtuel s'ouvre, l'interface glisse proprement vers le haut, éliminant tout crash visuel d'affichage (Keyboard Overflow).
- **Design soigné** : Les champs de saisie et le bouton bénéficient d'ombres colorées asymétriques orientées exclusivement vers la bordure basse (`BoxShadow` avec `Offset(0, 4)`) pour un effet de relief élégant et épuré.
- **Gestion déportée des erreurs** : Les messages de validation s'affichent nativement sous l'ombre des conteneurs, préservant la structure graphique et évitant de décaler le texte saisi à l'intérieur.
- **En cas d'échec** : Affichage d'un message explicite via une `SnackBar` rouge : « Email ou mot de passe incorrect ».

1.2 Écran principal : liste des notes

- **Affichage moderne** : Présentation des tâches sous forme de tuiles blanches épurées dotées, elles aussi, d'une ombre portée uniquement sur la bordure basse (`Offset(0, 4)`) pour se détacher proprement du fond de l'application.

- **Bouton d'ajout** : Un champ supérieur clair accompagné d'un bouton + indigo, facilement identifiable, pour l'insertion instantanée d'une nouvelle note.
- **Actions rapides** : Pour chaque note, l'utilisateur dispose d'une icône crayon (orange) pour modifier et d'une icône poubelle (rouge) pour supprimer.

1.3 Édition des notes

L'édition repose sur une boîte de dialogue modale et ergonomique dotée d'un champ pré-rempli avec le texte actuel de la note.

- **Boutons d'action clairs** : Annuler (orange) / Enregistrer (indigo).
- **Persistance** : Sauvegarde transparente en base locale SQLite avec rafraîchissement immédiat de la vue graphique.

1.4 Suppression des notes

Une fenêtre de dialogue pour confirmation avant suppression définitive est implantée afin d'éviter les erreurs de manipulation ou les clics accidentels.

- **Boutons d'action** : Annuler (indigo) / Supprimer (rouge).

1.5 Déconnexion sécurisée

Une icône logout est placée de manière très accessible dans la barre d'outils (AppBar).

L'application utilise la méthode `Navigator.pushReplacement` pour détruire l'historique complet de la session, interdisant ainsi tout retour involontaire ou frauduleux vers la liste des tâches sans s'authentifier au préalable.

2. Base de données (SQLite)

Le stockage est entièrement géré localement dans un fichier persistant sécurisé nommé : `todo_list.db`

2.1 Structure des Tables

Table : users

Champ	Type	Description
id	INTEGER	Clé primaire (PK) auto-incrémentée

email	TEXT	Adresse de messagerie de l'utilisateur
password	TEXT	Mot de passe d'authentification

Table : todo_list

Champ	Type	Description
id	INTEGER	Clé primaire (PK) auto-incrémentée
tache	TEXT	Contenu textuel de la note / tâche

Sécurité & Initialisation au démarrage

Au tout premier lancement de l'application (bloc onCreate), un utilisateur administrateur unique est injecté en base via le modèle User.admin(). L'utilisation de la règle ConflictAlgorithm.ignore garantit qu'aucun doublon ni conflit ne sera créé lors des démarrages ultérieurs.

2.2 Operations CRUD implantées

- **loginUser(email, password)** : Vérification des identifiants et conversion automatique de la ligne SQL via le constructeur User.fromMap.
- **insertTache(todo)** : Insertion d'une nouvelle tâche en base locale.
- **getAllTaches()** : Récupération de l'ensemble des tâches pour génération dynamique de la liste.
- **updateTache(todo)** : Mise à jour d'une tâche modifiée par l'utilisateur.
- **deleteTache(id)** : Suppression définitive d'un élément par son identifiant.

3. Architecture & Organisation du projet

Le code source respecte une architecture modulaire, découplée et conforme aux bonnes pratiques Flutter :

```

lib/
├── database/
│   └── database_manager.dart # Initialisation de SQLite (Batch, Tables, Seed Admin)
├── model/
│   ├── user.dart           # Modèle Utilisateur (Mapping Map/Object & Admin Factory)
│   └── todo_list.dart      # Modèle Note/Tâche (Mapping SQLite et Auto-incrément)
└── views/
    ├── login_interface.dart # Interface de connexion scrollable avec ombres et erreurs déportées
    └── todo_list_interface.dart # Écran principal CRUD (Liste avec ombres basses, Modals et Logout)

```

4. Installation & Procédure de Déploiement (Dev)

4.1 Prérequis système

- Flutter version stable
- Android Studio / Xcode configurés
- Émulateur configuré ou terminal physique avec débogage USB activé

4.2 Commandes d'installation

```

git clone https://github.com/Rachad-ac/projet_final_dclic.git
cd app_to_do_list
flutter doctor
flutter pub get

```

4.3 Exécution

```

flutter devices
flutter run

```

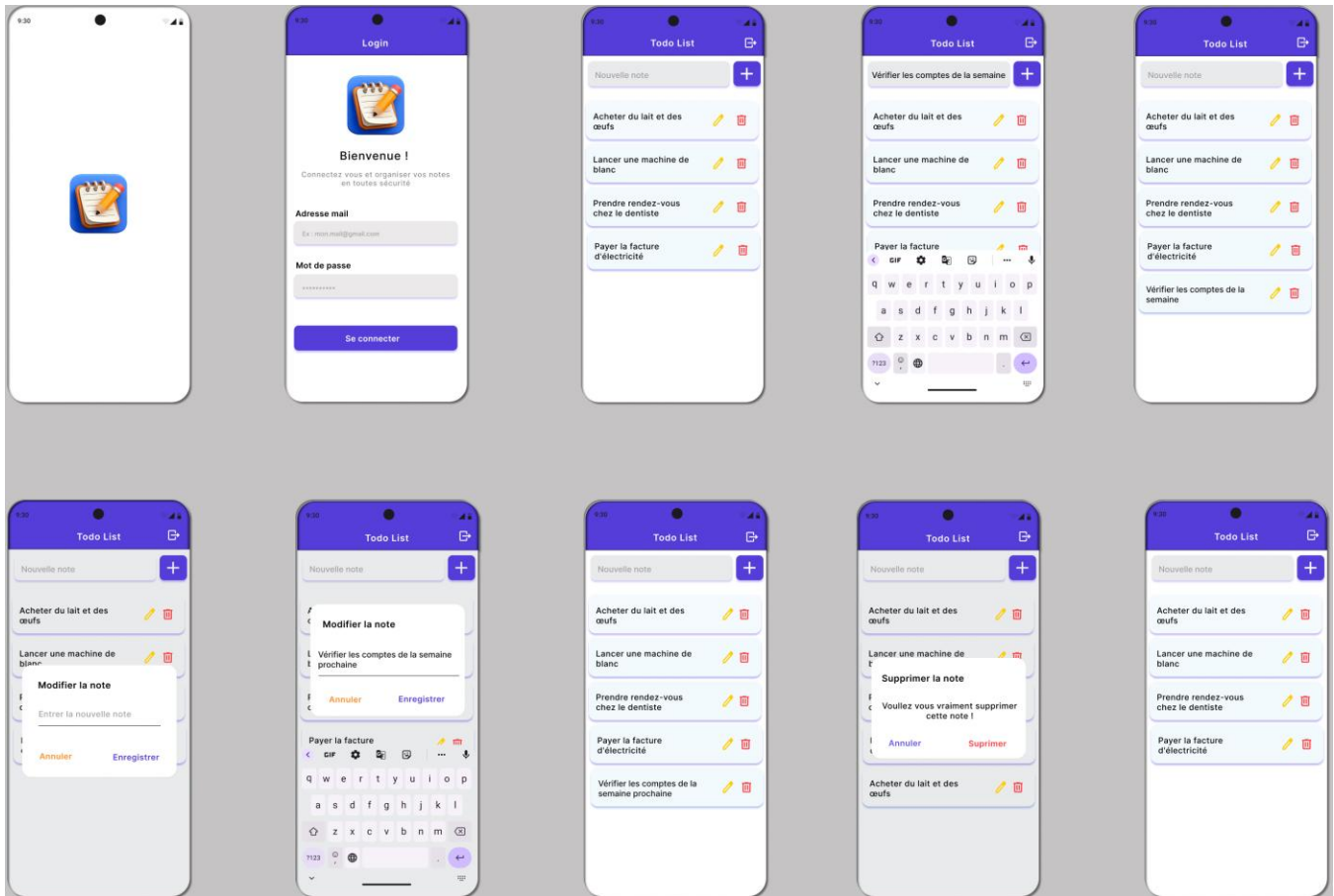
5. Parcours Utilisateur & Validation

- **Démarrage** : L'application affiche le Splash natif personnalisé avec le logo pendant le chargement initial et l'éveil de SQLite.
- **Authentification** : Saisie des identifiants sur l'écran dédié.
Identifiant par défaut : admin@gmail.com
Mot de passe : password1234

- **3. Contrôle d'accès** : En cas d'erreur, une Snackbar explicite apparaît. Si les identifiants sont corrects, l'accès est accordé et l'écran de login est détruit de la pile de navigation.
- **4. Gestion CRUD** : Ajout (+), édition (crayon) et suppression (poubelle avec confirmation visuelle).
- **5. Session** : Déconnexion via l'icône de la barre supérieure (AppBar) avec sécurisation des routes.

6. Maquettes & Ressources Externes

- **Lien Figma de conception** : <https://www.figma.com/design/MilySsiE6xFTPqEjWrFNWw/gestion-de-notes?node-id=0-1&t=IDmpOTpUWM2shM5H-1>



- **Lien Github** : https://github.com/Rachad-ac/projet_final_dcllic.git
- **Dossier Local** : Les maquettes et les wireframes originaux ayant servi de référence pour l'intégration visuelle sont disponibles directement dans le dossier interne : /assets/wireframes/

7. Crédits

Projet de fin de formation de la Semaine 6 réalisé dans le cadre de la formation développement mobile (Flutter) - Niveau Intermédiaire - DCLIC « Formez-vous au numérique avec l'OIF ».